asknget Backend

Developer Guide for Frontend Engineers and Stakeholders

Version 0.2 | 2026-09-13 | Django 6.1 + django-ninja 1.7 | API base: `/api/v1/`

This guide explains how the asknget backend is put together, which API to call at each step of the user journeys, the technologies involved, and how to run it locally with **uv**. It is written for the frontend team first, but the overview and flow sections are intended to be readable by product and QA stakeholders too.

Note. The project runs on an agile cadence and the Figma design changes frequently. This document reflects the design snapshot of 2026-09-11 and the code on branch `main`. Live, always-current API documentation is served by the backend itself at `/api/v1/docs` (Swagger UI). When this PDF and Swagger disagree, Swagger wins.

Contents

1. What the backend serves 3

Delivery status . . . 3

2. Technology stack 4

3. Architecture 4

Repository layout . . . 4

The three-file pattern inside every app . . . 4

Request lifecycle . . . 5

Design principles the frontend can rely on . . . 5

4. Running the backend locally with uv 6

Prerequisites . . . 6

First-time setup . . . 6

Environment variables (backend/.env) . . . 6

Useful URLs once running . . . 6

Everyday commands . . . 7

5. Authentication and error handling 8

Tokens . . . 8

Session lifecycle the frontend must implement . . . 8

Status codes and error shape . . . 8

6. API flows by user journey 9

6.1 Customer sign-up and verification . . . 9

6.2 Login . . . 9

6.3 My Account - Personal Information . . . 10

6.4 My Account - Password Manager and Logout . . . 10

6.5 My Account - Manage Addresses . . . 10

6.6 My Account - Payment Methods . . . 10

6.7 Seller sign-up and KYC application (planned) . . . 11

6.8 Storefront: browse, cart, checkout, orders (planned) . . . 12

7. Endpoint reference (implemented today) 13

Key schemas . . . 13

8. Data conventions for the frontend 14

9. Working with the API day to day 14

Trying calls in Swagger . . . 14

Generating a typed client . . . 14

Where to look when something is unclear . . . 14

Roadmap (subject to sprint planning) . . . 15

1. What the backend serves

asknget is a **multi-vendor e-commerce marketplace**. The backend exposes one JSON API that serves three audiences:

- **Customers** - browse the catalogue, manage cart, checkout, track orders, maintain profile, addresses, saved cards and wishlist.
- **Sellers (vendors)** - sign up, complete a 5-step KYC application (personal, business, representative, banking, review), upload documents, respond to reviewer requests, and activate a store once approved.
- **Admins** - review KYC applications, request more information, approve or reject, and manage catalogue data. Admin screens are not in the design yet; the Django admin site covers this for now.

Delivery status

Area	Status	Notes
Data model (38 tables) + migrations	Implemented	Derived from all 38 Figma screens; see docs/data-model.md
Django admin for every model	Implemented	http://127.0.0.1:8000/admin/
Auth: register, login, JWT, refresh, logout	Implemented	/api/v1/auth/*
OTP verification (email, SMS), password change/reset	Implemented	SMS provider is a console stub until an SMS gateway is chosen
Profile, avatar, addresses, payment methods	Implemented	/api/v1/me/*
Seller application, documents, status, store activation	Planned	Next sprint; models exist, endpoints do not yet
Catalogue: categories, products, filters, PDP, reviews	Planned	Models exist
Cart, vouchers, checkout, orders, tracking	Planned	Models exist; payment provider TBD
Banners, newsletter, contact form	Planned	Models exist
Vendor sub-roles / permissions, premium tier, vendor-customer chat	Not designed	Requested by team lead; not in Figma; schema proposal pending

2. Technology stack

Layer	Technology	Why / notes
Language	Python 3.13	Pinned in backend/.python-version
Package / env manager	uv 0.12	Replaces pip + venv; lockfile uv.lock guarantees identical installs
Web framework	Django 6.1	ORM, migrations, admin, auth primitives
API framework	django-ninja 1.7	Fast, typed REST on top of Django; pydantic v2 validation; auto OpenAPI/Swagger
Validation	pydantic 2	Every request body is a typed schema; errors are returned as 422 with field paths
Auth	PyJWT (HS256)	Short-lived access tokens; opaque rotating refresh tokens stored hashed
Database	PostgreSQL (target) / SQLite (dev fallback)	Set DATABASE_URL to switch; psycopg 3 driver
Field encryption	cryptography (Fernet)	ID/passport numbers and bank account numbers encrypted at rest
Images	Pillow	Avatar, product, banner, store images
Config	python-dotenv + dj-database-url	All settings via environment variables / backend/.env
Docs	Swagger UI (built in)	/api/v1/docs and /api/v1/openapi.json
Tests	Django TestCase	End-to-end through the real URL config; 29 tests for accounts

3. Architecture

Repository layout

```
asknget/                    git root  (github.com/AunSyedShah/asknget)
+-- docs/
|   +-- data-model.md       every table, field, enum, relationship, open questions
|   +-- figma/snapshot.py   dumps the Figma screens as text; --check detects design drift
|   +-- build_developer_guide.py   generates this PDF
+-- backend/
    +-- pyproject.toml, uv.lock, .env.example, README.md
    +-- asknget/            Django project root  (manage.py lives here)
        +-- asknget/        settings.py, urls.py, api.py (NinjaAPI root, error handlers)
        +-- core/           shared: base models, EncryptedTextField, validators, JWT auth, seed commanc
        +-- accounts/       User, OTP, refresh tokens, terms audit, addresses, payment methods
        +-- catalog/        categories, brands, products, variants/options, reviews, wishlist
        +-- sellers/        seller application (KYC), documents, info requests, stores
        +-- orders/         cart, vouchers, delivery methods, orders, status timeline, payments
        +-- marketing/      banners, newsletter, contact messages
```

The three-file pattern inside every app

Each Django app is split the same way so that a change lands in exactly one place. This is the main reason the codebase tolerates frequent design changes:

File	Owns	Changes when...
schemas.py	Request/response shapes and field-level validation (pydantic)	a form on the design changes
services.py	Business rules: uniqueness, cooldowns, state machines, side effects, transactions	a flow or rule changes
api.py	Routers - thin functions: parse, call a service, return	an endpoint is added or renamed
models.py	Database tables and constraints	the data model changes (needs a migration)

File	Owns	Changes when...
tests.py	End-to-end tests through the real URLs	any of the above changes

Request lifecycle

```
Frontend  --HTTP JSON-->  Django URLconf (/api/v1/)  -->  NinjaAPI
                                                          |
                              1. Auth class (JWTAuth) validates Bearer token -> request.auth = User
                              2. pydantic schema validates + normalises the body (422 on failure)
                              3. Router function calls a service
                              4. Service enforces rules, touches the ORM inside a transaction
                              5. Response schema serialises the result (explicit field list only)
                                                          |
Frontend  <--JSON + status code (200/201/204 or 4xx {"detail": ...})--
```

Design principles the frontend can rely on

- **Explicit responses.** Every response schema lists its fields; nothing leaks by accident (no password hashes, no internal tokens).
- **Separate In / PatchIn / Out schemas.** Create payloads require their fields; PATCH payloads are fully optional and *omitted is not null* - only keys you send are updated.
- **Enums are shared.** Dropdown values (address type, gender, order status, application status...) come from one source and appear in Swagger.
- **Rules live server-side.** The frontend should validate for UX, but the API is the authority (cooldowns, uniqueness, default switching, state transitions).
- **Versioned base path.** Everything is under `/api/v1/`. A breaking change means a new version, not an edit.

4. Running the backend locally with uv

Prerequisites

- Git, and **uv** (https://docs.astral.sh/uv/). uv installs the pinned Python 3.13 for you; no system Python required.
- Optional: PostgreSQL 15+ if you want parity with production. Without it the project falls back to SQLite automatically.

First-time setup

```
git clone https://github.com/AunSyedShah/asknget.git
cd asknget/backend

uv sync                                  # creates .venv and installs exactly what uv.lock says
cp .env.example .env                     # then edit .env (see below)

cd asknget                               # the Django project root (contains manage.py)
uv run python manage.py migrate          # create the database schema
uv run python manage.py seed_lookups     # categories, brands, banks, delivery methods
uv run python manage.py createsuperuser  # admin login for http://127.0.0.1:8000/admin/
uv run python manage.py runserver        # http://127.0.0.1:8000
```

Environment variables (backend/.env)

Variable	Required	Purpose
SECRET_KEY	prod	Django secret; also signs access tokens. A dev default exists but never use it beyond localhost.
DEBUG	no	1 (default) or 0
ALLOWED_HOSTS	prod	Comma-separated hostnames
DATABASE_URL	no	postgres://user:pass@host:5432/asknget - omit to use SQLite (backend/asknget/db.sqlite3)
FIELD_ENCRYPTION_KEY	yes*	Fernet key for encrypted PII. *Required before any seller banking/ID data is saved. Generate: python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
JWT_ACCESS_TTL_MINUTES	no	Default 15
JWT_REFRESH_TTL_DAYS	no	Default 30
OTP_TTL_MINUTES / OTP_RESEND_COOLDOWN_SECONDS / OTP_MAX_ATTEMPTS	no	Defaults 10 / 45 / 5

Useful URLs once running

URL	What
http://127.0.0.1:8000/api/v1/docs	Swagger UI - try every endpoint in the browser (use Authorize with a Bearer token)
http://127.0.0.1:8000/api/v1/openapi.json	OpenAPI 3 spec - feed it to a client generator (openapi-typescript, orval, etc.)
http://127.0.0.1:8000/api/v1/health	Liveness probe
http://127.0.0.1:8000/admin/	Django admin - inspect and edit any table

Everyday commands

```
uv run python manage.py test                # run all tests
uv run python manage.py test accounts       # one app
uv run python manage.py makemigrations      # after changing models
uv run python manage.py migrate
uv add <package>                            # add a dependency (updates pyproject + uv.lock)
python ../docs/figma/snapshot.py --check    # has the Figma design changed since we last looked?
```

Note. OTP emails are printed to the runserver console in development (console email backend); SMS codes are logged. Look at the terminal running `runserver` to find the 6-digit code while testing.

5. Authentication and error handling

Tokens

Token	Lifetime	Where to keep it	How it is used
access_token (JWT)	15 min	Memory (not localStorage if you can avoid it)	Header on every protected call: Authorization: Bearer <access_token>
refresh_token (opaque)	30 days	Secure httpOnly cookie or secure storage	POST /auth/refresh to get a new pair. It is single-use: every refresh returns a new refresh token and the old one is dead.

Session lifecycle the frontend must implement

#	Screen / user action	API call	Backend behaviour
1	App start	POST /auth/refresh {refresh_token}	Rotates the token; 401 means the user must log in again
2	Any protected call returns 401 'Access token expired'	POST /auth/refresh, then retry the original call once	Standard silent-refresh interceptor
3	User presses Logout (popup)	POST /auth/logout {refresh_token} or {all_devices: true}	Revokes the refresh token(s). Discard both tokens client-side.
4	Password changed or reset	-	All refresh tokens are revoked server-side; other devices are signed out on their next refresh

Status codes and error shape

Code	Meaning	Body
200 / 201 / 204	OK / created / no content	Resource, or empty for 204
400	Business rule failed on user input (wrong OTP, wrong current password)	{"detail": "..."}
401	Missing/invalid/expired token, or bad credentials	{"detail": "..."}
403	Account suspended / not allowed	{"detail": "..."}
404	Resource not found or not yours	{"detail": "Not found"}
409	Conflict - email or phone already registered	{"detail": "..."}
422	Validation failed	{"detail": "summary", "errors": [{"field": "phone", "message": "..."}]}
429	Rate limited - OTP resend cooldown or too many attempts	{"detail": "Please wait 32s ..."}

Example 422 body:

```
{
  "detail": "phone: Phone must be in international format, e.g. +923001234567;
             confirm_password: Passwords do not match",
  "errors": [
    {"field": "phone", "message": "Phone must be in international format, e.g. +923001234567"},
    {"field": "confirm_password", "message": "Passwords do not match"}
  ]
}
```

Use `errors[].field` to highlight inputs and `detail` as a fallback toast. A field of `null` means the error concerns the whole payload.

6. API flows by user journey

Each table reads top to bottom: what the user does on the screen, which API the frontend calls, and what the backend does. Endpoints marked *planned* are designed but not yet implemented.

6.1 Customer sign-up and verification

#	Screen / user action	API call	Backend behaviour
1	Sign-up form submitted	POST /auth/register {email, password, confirm_password, phone?, full_name?}	Creates the user, returns user + tokens (201). Email lower-cased, phone normalised to E.164. 409 if taken.
2	'Verify Your Email' screen shown	POST /auth/otp/send {channel: 'email', purpose: 'signup_email'} (Bearer)	Sends 6-digit code; response has target_masked, expires_at, resend_available_at (45 s cooldown)
3	User types the 6 digits	POST /auth/otp/verify {channel, purpose, target: email, code}	Marks email_verified_at. 400 wrong code, 429 after 5 wrong attempts.
4	'Resend code in 00:45' countdown ends	POST /auth/otp/send again	429 with remaining seconds if called too early
5	Mobile OTP screen (same pattern)	channel: 'sms', purpose: 'signup_phone'	Sets phone_verified_at

Note. Registration already returns tokens, so the user is logged in immediately; there is no need to call /auth/login after sign-up. The `user` object in the response is the same shape as `GET /me`, with profile fields empty until the user fills them in on the Personal Information page.

6.2 Login

#	Screen / user action	API call	Backend behaviour
1	Sign-in form	POST /auth/login {email, password}	200 with user + tokens; 401 bad credentials; 403 suspended
2	'Forgot password'	POST /auth/password/forgot {email}	Always 200 (does not reveal whether the email exists); emails a code if it does
3	Reset form	POST /auth/password/reset {email, code, new_password, confirm_password}	Sets password, revokes all sessions. User must sign in again.

6.3 My Account - Personal Information

#	Screen / user action	API call	Backend behaviour
1	Page opens	GET /me	Full profile incl. email_verified, phone_verified, member_since (Account Information panel)
2	'Save Changes' on Personal Details	PATCH /me {full_name?, gender?, date_of_birth?, profession?, country?, city?, address_line?}	Partial update - only sent keys change. Enums and past-date checks return 422.
3	'Change Photo'	POST /me/avatar (multipart, field name 'file')	PNG/JPG/GIF, max 2 MB; returns updated profile with avatar_url
4	Change email or phone	OTP flow (6.1) - not PATCH /me	Email/phone are identity fields and require re-verification (endpoint planned)

6.4 My Account - Password Manager and Logout

#	Screen / user action	API call	Backend behaviour
1	'Update Password'	POST /me/password {current_password, new_password, confirm_password}	400 if current is wrong; on success all other sessions are signed out
2	Logout popup confirmed	POST /auth/logout {refresh_token}	204; clear tokens client-side

6.5 My Account - Manage Addresses

#	Screen / user action	API call	Backend behaviour
1	Tab opens	GET /me/addresses	List, default first
2	'Save Address' in popup	POST /me/addresses {type, label?, full_name, phone, street_address, unit_floor?, city, province?, postal_code, country?, delivery_instructions?, is_default?}	201. The first address always becomes default; is_default=true moves the default.
3	'Edit' on a card	PATCH /me/addresses/{id}	Partial update
4	Set as default	POST /me/addresses/{id}/default	Exactly one default is enforced by a DB constraint
5	Delete	DELETE /me/addresses/{id}	204; soft-deleted; if it was default, the newest remaining address becomes default

6.6 My Account - Payment Methods

Note. Card numbers and CVV never reach this API. The frontend tokenises the card with the payment provider's SDK and sends the token plus display data. The provider is not chosen yet; the contract below is provider-agnostic.

#	Screen / user action	API call	Backend behaviour
1	Tab opens	GET /me/payment-methods	List with display_name like 'VISA ....2389'

#	Screen / user action	API call	Backend behaviour
2	'Add Card' in popup	POST /me/payment-methods {type: 'card', provider, provider_token, card_brand, card_holder_name, last4, exp_month, exp_year, is_default?}	201. For type=card all card fields are required and expiry must be in the future. PayPal/Apple/Google Pay need only type, provider, provider_token.
3	Set default / delete	POST .../{id}/default, DELETE .../{id}	Same semantics as addresses

6.7 Seller sign-up and KYC application (planned)

#	Screen / user action	API call	Backend behaviour
1	'Create your seller account' (account type radio + form + terms checkbox)	POST /auth/seller/register {email, phone, password, confirm_password, account_type, accept_terms: true}	Implemented. Creates user (is_seller) + draft application + terms audit rows; returns tokens. 422 if accept_terms is false.
2	Email and mobile OTP screens	Same as 6.1	Implemented
3	Stepper steps 1-4, 'Save & Continue'	PUT /seller/application/step/{1..4} (planned)	Autosaves the step; validates only that step; advances current_step
4	'+ Upload Document' (ID, company registration, bank letter, proof of address)	POST /seller/application/documents (multipart, type) (planned)	PDF/JPG/PNG up to 10 MB; versions kept
5	'+ Add Another Director'	POST /seller/application/directors (planned)	
6	Step 5 'Submit'	POST /seller/application/submit (planned)	Requires both confirmation checkboxes; status draft -> submitted; issues application number ASK-SLR-xxxxxx
7	Application Status page	GET /seller/application/status (planned)	Timeline: submitted -> under_review -> (more_info_required | approved | rejected)
8	'Update & Resubmit' after reviewer request	POST .../documents/{id}/replace, PUT step, POST /seller/application/resubmit (planned)	Resolves the open info request; back to under_review
9	'Activate & Start Selling'	POST /seller/store/activate {display_name, slug, agreements} (planned)	Creates/activates the store; slug becomes asknget.com/store/<slug>

6.8 Storefront: browse, cart, checkout, orders (planned)

#	Screen / user action	API call	Backend behaviour
1	Landing page	GET /banners?placement=hero|top_discount, GET /products/featured|best-sellers|new-arrivals, GET /categories	Public
2	Categories page with filters	GET /products?category=&brand;[]=&color;[]=&price;_min=&price;_max=∈_stock=&q;=&sort;=&page;=&per;_page=12	Returns {items, total, page}; 12 per page as in the design
3	Product page tabs	GET /products/{slug}, GET /products/{id}/reviews, GET /products/{id}/related	Options (Size/Shade/Color), variants, benefits, how-to-use steps
4	'Submit Review'	POST /products/{id}/reviews (multipart)	Verified Purchase badge when tied to an order item
5	Add to cart / quantity	GET /cart, POST /cart/items, PATCH|DELETE /cart/items/{id}	Works for guests via a cart session token
6	'Apply' voucher, order note	POST /cart/voucher, PATCH /cart {note}	Recomputes discount
7	Checkout 'Place Order'	GET /delivery-methods, POST /checkout	Creates the order + payment intent; snapshots address, prices, delivery method
8	Order Placed / Track Order	POST /orders/track {order_number, email} (public)	Status timeline
9	My Orders tabs	GET /me/orders?status=ongoing|completed|cancelled, POST /me/orders/{id}/cancel, POST /me/orders/{id}/reorder	Tabs map to status groups
10	Wishlist heart	PUT|DELETE /me/wishlist/{productId}, GET /me/wishlist	Implemented model; endpoint planned

7. Endpoint reference (implemented today)

All paths are prefixed with `/api/v1`. 'Auth' = requires `Authorization: Bearer`.

Method + path	Auth	Request body	Success	Errors
POST /auth/register	-	RegisterIn	201 AuthOut	409, 422
POST /auth/seller/register	-	SellerRegisterIn	201 AuthOut	409, 422
POST /auth/login	-	LoginIn	200 AuthOut	401, 403
POST /auth/refresh	-	RefreshIn	200 TokenOut	401
POST /auth/logout	yes	LogoutIn	204	401
POST /auth/otp/send	optional	OtpSendIn	200 OtpSendOut	422, 429
POST /auth/otp/verify	-	OtpVerifyIn	200 MessageOut	400, 429
POST /auth/password/forgot	-	PasswordForgotIn	200 MessageOut	429
POST /auth/password/reset	-	PasswordResetIn	200 MessageOut	400, 422
GET /me	yes	-	200 UserOut	401
PATCH /me	yes	UserPatchIn	200 UserOut	422
POST /me/avatar	yes	multipart file	200 UserOut	422
POST /me/password	yes	PasswordChangeIn	200 MessageOut	400, 422
POST /me/terms	yes	TermsAcceptIn	201 MessageOut	
GET /me/addresses	yes	-	200 [AddressOut]	
POST /me/addresses	yes	AddressIn	201 AddressOut	422
GET /me/addresses/{id}	yes	-	200 AddressOut	404
PATCH /me/addresses/{id}	yes	AddressPatchIn	200 AddressOut	404, 422
POST /me/addresses/{id}/default	yes	-	200 AddressOut	404
DELETE /me/addresses/{id}	yes	-	204	404
GET /me/payment-methods	yes	-	200 [PaymentMethodOut]	
POST /me/payment-methods	yes	PaymentMethodIn	201 PaymentMethodOut	422
POST /me/payment-methods/{id}/default	yes	-	200 PaymentMethodOut	404
DELETE /me/payment-methods/{id}	yes	-	204	404
GET /health	-	-	200 {status: ok}	

Key schemas

```
UserOut {
  id: uuid, email, phone, full_name, first_name, last_name,
  gender: "male"|"female"|"other"|"prefer_not_to_say"|"",
  date_of_birth: "YYYY-MM-DD"|null, profession, country: "PK", city, address_line,
  status: "active"|"suspended"|"deleted", is_seller, is_staff,
  email_verified: bool, phone_verified: bool, member_since: ISO datetime, avatar_url|null
}
TokenOut  { access_token, refresh_token, token_type: "bearer", expires_in: 900 }
AuthOut   { user: UserOut, tokens: TokenOut }
AddressOut { id, type: "home"|"office"|"other", label, full_name, phone, street_address, unit_floor,
             city, province, postal_code, country, delivery_instructions, is_default, created_at, updated_at }
PaymentMethodOut { id, type: "card"|"paypal"|"apple_pay"|"google_pay"|"bank_transfer",
                   card_brand: "visa"|"mastercard"|"other"|"", card_holder_name, last4,
                   exp_month, exp_year, provider, is_default, display_name, created_at }
```

8. Data conventions for the frontend

Topic	Convention
IDs	UUID strings everywhere (users, addresses, products, orders...)
Dates/times	ISO 8601 in UTC, e.g. 2026-09-12T20:39:00Z. Dates without time are YYYY-MM-DD. Format for display client-side.
Money	Integers in minor units: price_cents = 2599 means $25.99, alongside a currency code. Never send floats.
Phone numbers	E.164 (+923001234567). The API accepts spaces/dashes/parentheses and normalises; display formatting is the client's job.
Email	Lower-cased by the server
Enums	Lower-snake_case strings exactly as listed in Swagger. Unknown values are rejected with 422.
Optional text fields	Come back as "" (empty string) when unset, not null; nullable dates/numbers come back as null.
Partial updates	Use PATCH and send only the keys that changed. Sending null clears a field.
Lists	Small lists (addresses, cards) are plain arrays. Catalogue/orders will be paginated: {items, total, page, per_page} with per_page 12 by default.
File uploads	multipart/form-data with the field named 'file' (avatar) or per the endpoint; size/type limits are enforced server-side and returned as 422.
Idempotency	Setting a default twice, or verifying an already-verified email, is safe.

9. Working with the API day to day

Trying calls in Swagger

- Open /api/v1/docs, expand **POST /auth/register**, 'Try it out', submit. Copy `tokens.access_token` from the response.
- Click **Authorize** (top right), paste the access token (without 'Bearer '), then any /me endpoint works.
- For OTP codes, look at the terminal running `runserver`.

Generating a typed client

```
# TypeScript types straight from the live spec
npx openapi-typescript http://127.0.0.1:8000/api/v1/openapi.json -o src/api/schema.d.ts
```

Regenerate the client whenever the backend announces a schema change; the spec is the contract.

Where to look when something is unclear

Question	Source
What fields does endpoint X accept and return?	Swagger /api/v1/docs (authoritative), then backend/asknget/<app>/schemas.py
Why did the API reject my payload?	The 422 body's errors[]; rules are in schemas.py, business rules in services.py
Which tables/fields exist and why?	docs/data-model.md (every table annotated with the Figma label it came from)
Has the design changed since the backend was built?	python docs/figma/snapshot.py --check, then git diff docs/figma/
Open product questions (currency, guest checkout, multi-vendor shipping...)	docs/data-model.md section 10

Roadmap (subject to sprint planning)

Order	Scope	Depends on
1	Sellers: application steps, documents, submit/resubmit, status timeline, store activation; admin review endpoints	Auth (done)
2	Catalogue read APIs: categories, product listing with filters, PDP, reviews, wishlist	Seller stores (products belong to stores)
3	Cart, vouchers, delivery methods, checkout, orders, tracking; payment provider integration	Catalogue; provider decision
4	Marketing: banners, newsletter, contact	-
5	Vendor sub-roles/permissions, premium tier, vendor-customer messaging	Product decisions + design

Repository: https://github.com/AunSyedShah/asknget | Backend README: backend/README.md | This document is generated by docs/build_developer_guide.py - edit the script, not the PDF.